

Go for Senior .NET Engineers

Phase 4: Standard Library & Ecosystem

9 Lessons: HTTP, JSON, I/O, Testing, SQL, Templates, File I/O, Logging, Time

Current as of Go 1.25.6 (January 2026) — includes ServeMux routing, json/v2, log/slog

Lesson 34: net/http — HTTP Server & Client

Go's net/http is production-grade out of the box. Since Go 1.22, the standard ServeMux supports method-based routing and path parameters — eliminating the need for third-party routers like gorilla/mux or chi for most APIs.

34.1 — Minimal HTTP Server (Go 1.22+ Routing)

```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

func main() {
    mux := http.NewServeMux()

    // Method + path params (Go 1.22+)
    mux.HandleFunc("GET /users/{id}", getUser)
    mux.HandleFunc("POST /users", createUser)
    mux.HandleFunc("DELETE /users/{id}", deleteUser)
    mux.HandleFunc("GET /health", healthCheck)

    fmt.Println("listening on :8080")
    http.ListenAndServe(":8080", mux)
}

func getUser(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id") // Go 1.22+ path param
    json.NewEncoder(w).Encode(map[string]string{
        "id": id, "name": "Alice",
    })
}
```

34.2 — C# ASP.NET Minimal API vs Go net/http

C# / .NET

```
// C# – ASP.NET Minimal API
var app = WebApplication.Create();

app.MapGet("/users/{id}",
    (int id) => new User(id));

app.MapPost("/users",
    (User u) => Results.Created(
        $"/users/{u.Id}", u));

app.Run();
```

Go

```
// Go – net/http (Go 1.22+)
mux := http.NewServeMux()

mux.HandleFunc("GET /users/{id}",
    getUser)

mux.HandleFunc("POST /users",
    createUser)

http.ListenAndServe(":8080", mux)
```

34.3 — HTTP Client

```
// Create a client with timeout (ALWAYS set a timeout)
client := &http.Client{
    Timeout: 10 * time.Second,
}

// GET request
resp, err := client.Get("https://api.example.com/users")
if err != nil {
    return fmt.Errorf("request failed: %w", err)
}
defer resp.Body.Close()

// POST with JSON body
body, _ := json.Marshal(user)
resp, err := client.Post(
    "https://api.example.com/users",
    "application/json",
    bytes.NewReader(body),
)
```

34.4 — Middleware Pattern

```
// Middleware = func(http.Handler) http.Handler
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        slog.Info("request",
            "method", r.Method,
            "path", r.URL.Path,
            "duration", time.Since(start),
        )
    })
}

// Chain: http.ListenAndServe(":8080", Logger(mux))
```

WARNING: ALWAYS set `http.Client.Timeout`. The default client has NO timeout, so a slow server will block your goroutine forever. Also ALWAYS call `resp.Body.Close()` via `defer`.

Lesson 35: encoding/json — Marshal, Unmarshal & JSON v2

Go's JSON handling is struct-tag-driven, similar to System.Text.Json attributes. Go 1.25 introduced an experimental encoding/json/v2 package with major performance and correctness improvements (2-10x faster unmarshaling, stricter parsing, better error messages).

35.1 — Basic Marshal / Unmarshal

C# / .NET

```
// C# - System.Text.Json
var json = JsonSerializer
    .Serialize(user);

var user = JsonSerializer
    .Deserialize<User>(json);

// Attributes
[JsonPropertyName("first_name")]
public string FirstName { get; set; }

[JsonIgnore]
public string Password { get; set; }
```

Go

```
// Go - encoding/json
data, err := json.Marshal(user)

var user User
err := json.Unmarshal(data, &user)

// Struct tags
type User struct {
    FirstName string `json:"first_name"`
    Password  string `json:"-"`
    Age       int
    `json:"age,omitempty"`
}
```

35.2 — Common Struct Tag Options

Tag	Behavior
`json:"name"`	Rename field to 'name' in JSON
`json:"-"`	Skip field entirely
`json:",omitempty"`	Omit if zero value
`json:",omitzero"`	Omit if IsZero() returns true (Go 1.24+)
`json:",string"`	Encode number/bool as JSON string

35.3 — Streaming with Encoder/Decoder

```
// Encode directly to http.ResponseWriter
func getUser(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(user)
}

// Decode from request body
func createUser(w http.ResponseWriter, r *http.Request) {
    var user User
```

```
    if err := json.NewDecoder(r.Body).Decode(&user); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }
    // use user...
}
```

35.4 — encoding/json/v2 (Go 1.25 Experimental)

```
// Enable: GOEXPERIMENT=jsonv2 go build
// Or import directly (experimental, API may change):
import jsonv2 "encoding/json/v2"

// Key improvements over v1:
// - 2-10x faster unmarshaling
// - Case-sensitive matching by default
// - Duplicate keys rejected by default
// - Better error messages with JSON path
// - Functional options for customization

b, err := jsonv2.Marshal(user,
    jsonv2.OmitZeroStructFields(true),
    jsonv2.StringifyNumbers(true),
    jsonv2.WithIndent("  "),
)
```

TIP: json/v2 is experimental in Go 1.25 and expected to become stable in Go 1.26. When GOEXPERIMENT=jsonv2 is enabled, the existing encoding/json package uses the v2 engine internally, so all your existing code gets faster automatically.

Lesson 36: io.Reader & io.Writer — The Universal Streaming Interfaces

io.Reader and io.Writer are Go's most fundamental interfaces. Every I/O operation — files, HTTP bodies, network connections, compression, encryption — is built on them. Understanding these two interfaces is key to writing efficient, composable Go code.

36.1 — The Interfaces

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

// These are satisfied by:
// os.File, bytes.Buffer, strings.Reader,
// http.Response.Body, net.Conn, gzip.Writer,
// crypto/tls.Conn, bufio.Reader, and hundreds more
```

36.2 — C# Stream vs Go io.Reader

C# / .NET

```
// C# - Stream (abstract class)
using var stream = File.OpenRead("f.txt");
using var reader = new
StreamReader(stream);
string content = await reader
    .ReadToEndAsync();

// Copy between streams
await input.CopyToAsync(output);

// Chaining: decompress + decrypt
using var gzip = new GZipStream(
    stream, CompressionMode.Decompress);
```

Go

```
// Go - io.Reader (interface)
f, err := os.Open("f.txt")
defer f.Close()
content, err := io.ReadAll(f)

// Copy between readers/writers
io.Copy(dst, src)

// Chaining: decompress + decrypt
gz, _ := gzip.NewReader(f)
defer gz.Close()
```

36.3 — Composing Readers (Pipeline)

```
// Read gzipped JSON from HTTP response:
resp, _ := http.Get("https://api.example.com/data.gz")
defer resp.Body.Close()

// Chain: HTTP body → gzip decompressor → JSON decoder
gz, _ := gzip.NewReader(resp.Body)
defer gz.Close()
```

```
var data MyStruct
json.NewDecoder(gz).Decode(&data)

// Each layer is an io.Reader wrapping the previous one
// No buffering needed – data streams through
```

36.4 — Useful io Utilities

Function	Purpose
<code>io.Copy(dst, src)</code>	Stream src → dst (like <code>Stream.CopyTo</code>)
<code>io.ReadAll(r)</code>	Read entire reader into <code>[]byte</code>
<code>io.LimitReader(r, n)</code>	Read at most n bytes
<code>io.TeeReader(r, w)</code>	Read from r, write to w simultaneously
<code>io.MultiReader(r1, r2)</code>	Concatenate multiple readers
<code>io.MultiWriter(w1, w2)</code>	Write to multiple writers at once
<code>io.Pipe()</code>	Synchronous in-memory pipe (reader/writer)

TIP: Think of `io.Reader/Writer` as Go's equivalent of C# `Stream`, but as interfaces instead of abstract classes. Because they are interfaces, ANY type can implement them. This is why Go's I/O is so composable.

Lesson 37: testing — Table-Driven Tests, Benchmarks, Fuzzing

Go has a built-in testing framework — no xUnit, NUnit, or MSTest needed. Tests live in `_test.go` files next to the code they test. The `go test` command discovers and runs them automatically.

37.1 — Basic Test

C# / .NET

```
// C# - xUnit
[Fact]
public void Add_ReturnsSum() {
    var result = Add(2, 3);
    Assert.Equal(5, result);
}

// Run: dotnet test
```

Go

```
// Go - testing package
func TestAdd(t *testing.T) {
    result := Add(2, 3)
    if result != 5 {
        t.Errorf("got %d, want 5",
            result)
    }
}

// Run: go test ./...
```

37.2 — Table-Driven Tests (Go's Signature Pattern)

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name      string
        a, b      int
        expected int
    }{
        {"positive", 2, 3, 5},
        {"negative", -1, -2, -3},
        {"zero", 0, 0, 0},
        {"mixed", -1, 5, 4},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got := Add(tt.a, tt.b)
            if got != tt.expected {
                t.Errorf("Add(%d,%d) = %d, want %d",
                    tt.a, tt.b, got, tt.expected)
            }
        })
    }
}
```

37.3 — Benchmarks

```
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(2, 3)
    }
}
```



```
    }  
}  
  
// Go 1.24+: use b.Loop() instead  
func BenchmarkAdd(b *testing.B) {  
    for b.Loop() {  
        Add(2, 3)  
    }  
}  
  
// Run: go test -bench=. -benchmem  
// Output: BenchmarkAdd-8 500000000 2.34 ns/op 0 B/op 0 allocs/op
```

37.4 — Fuzz Testing (Go 1.18+)

```
func FuzzReverse(f *testing.F) {  
    // Seed corpus  
    f.Add("hello")  
    f.Add("world")  
  
    f.Fuzz(func(t *testing.T, s string) {  
        rev := Reverse(s)  
        doubleRev := Reverse(rev)  
        if s != doubleRev {  
            t.Errorf("double reverse mismatch: %q", s)  
        }  
    })  
}  
  
// Run: go test -fuzz=FuzzReverse  
// Go generates random inputs and finds edge cases
```

TIP: Go testing philosophy: no assertion library needed. Use `if/t.Errorf` for simple assertions. The `testify` package (github.com/stretchr/testify) is popular for `assert.Equal`-style assertions, but many Go teams prefer the standard approach.

Lesson 38: database/sql — SQL Database Access

database/sql is Go's standard database interface, analogous to ADO.NET. It provides connection pooling, prepared statements, and transactions. You pair it with a driver (e.g., lib/pq for PostgreSQL, go-sql-driver/mysql for MySQL).

38.1 — C# ADO.NET vs Go database/sql

C# / .NET

```
// C# – ADO.NET / Dapper
using var conn = new NpgsqlConnection(
    connString);
await conn.OpenAsync();

var user = await conn
    .QueryFirstOrDefaultAsync<User>(
        "SELECT * FROM users WHERE id=@id",
        new { id = 42 });
```

Go

```
// Go – database/sql
db, err := sql.Open("postgres",
    connString)
defer db.Close()

var user User
err = db.QueryRowContext(ctx,
    "SELECT id, name, email FROM users
    WHERE id=$1",
    42).Scan(&user.ID, &user.Name,
    &user.Email)
```

38.2 — Connection Pooling

```
// sql.Open returns a pool, NOT a single connection
db, err := sql.Open("postgres", connString)

// Configure the pool
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(5)
db.SetConnMaxLifetime(5 * time.Minute)

// Ping to verify connectivity
if err := db.PingContext(ctx); err != nil {
    log.Fatal("cannot reach database:", err)
}

// db is safe for concurrent use – share it globally
```

38.3 — Queries and Scanning

```
// Multiple rows
rows, err := db.QueryContext(ctx,
    "SELECT id, name FROM users WHERE active=$1", true)
if err != nil { return err }
defer rows.Close()

var users []User
for rows.Next() {
    var u User
    if err := rows.Scan(&u.ID, &u.Name); err != nil {
```

```
        return err
    }
    users = append(users, u)
}
return rows.Err() // check iteration error
```

38.4 — Transactions

```
tx, err := db.BeginTx(ctx, nil)
if err != nil { return err }
defer tx.Rollback() // no-op if committed

_, err = tx.ExecContext(ctx,
    "UPDATE accounts SET balance=balance-$1 WHERE id=$2", amount, fromID)
if err != nil { return err }

_, err = tx.ExecContext(ctx,
    "UPDATE accounts SET balance=balance+$1 WHERE id=$2", amount, toID)
if err != nil { return err }

return tx.Commit()
```

WARNING: ALWAYS call `rows.Close()` (via `defer`) and check `rows.Err()` after iteration. Failing to close rows leaks database connections from the pool.

Lesson 39: text/template & html/template

Go's template packages provide logic-less templating with automatic HTML escaping (html/template). They are Go's equivalent of Razor Pages, but server-side only and much simpler.

39.1 — Basic Template Usage

```
import "html/template"

const tmpl = `

# Hello, {{.Name}}!</h1> <ul> {{range .Items}}<li>{{.}}</li> {{end}}</ul>` func renderPage(w http.ResponseWriter, data PageData) { t := template.Must(template.New("page").Parse(tmpl)) t.Execute(w, data) }


```

39.2 — Template Actions

Go Template	C# Razor Equivalent
{{.FieldName}}	@Model.FieldName
{{range .Items}} {{.}} {{end}}	@foreach (var item in Model.Items)
{{if .IsAdmin}} ... {{end}}	@if (Model.IsAdmin) { ... }
{{template "header" .}}	@await Html.PartialAsync("Header")
{{.Name toUpper}}	@Model.Name.ToUpper()

39.3 — Security: Auto-Escaping

```
// html/template auto-escapes all output:
// {{.UserInput}} with "<script>alert(1)</script>"
// renders as: &lt;script&gt;alert(1)&lt;/script&gt;

// To insert raw HTML (dangerous – trust the source):
// {{.TrustedHTML}} where TrustedHTML is template.HTML type
type PageData struct {
    UserInput    string    // auto-escaped
    TrustedHTML  template.HTML // NOT escaped
}
```

TIP: For modern web apps, many Go teams use html/template for server-rendered pages, or skip templates entirely and build a REST/gRPC API consumed by a React/Vue frontend. Go is not opinionated about this choice.

Lesson 40: os, filepath, bufio — File I/O

Go's file I/O is straightforward and low-level compared to C#'s System.IO. There are no FileStream wrappers — os.File implements io.Reader and io.Writer directly.

40.1 — Reading & Writing Files

C# / .NET

```
// C#
var text = await File.ReadAllTextAsync(
    "data.txt");

await File.WriteAllTextAsync(
    "output.txt", content);

var lines = await File
    .ReadAllLinesAsync("data.txt");
```

Go

```
// Go
text, err := os.ReadFile("data.txt")

err := os.WriteFile("output.txt",
    []byte(content), 0644)

// No ReadAllLines – use bufio.Scanner
```

40.2 — Line-by-Line Reading with bufio.Scanner

```
f, err := os.Open("large.log")
if err != nil { return err }
defer f.Close()

scanner := bufio.NewScanner(f)
for scanner.Scan() {
    line := scanner.Text() // current line (no newline)
    processLine(line)
}
return scanner.Err()
```

40.3 — filepath: Cross-Platform Paths

```
import "path/filepath"

// Join paths (handles OS separators)
path := filepath.Join("data", "users", "alice.json")

// Walk directory tree
filepath.WalkDir("./data", func(path string, d fs.DirEntry, err error) error {
    if err != nil { return err }
    if !d.IsDir() && filepath.Ext(path) == ".json" {
        processFile(path)
    }
    return nil
})

// Get parts
dir := filepath.Dir(path) // "data/users"
```

```
base := filepath.Base(path) // "alice.json"
ext  := filepath.Ext(path)  // ".json"
```

TIP: `os.ReadFile` and `os.WriteFile` (added in Go 1.16) handle the common case of reading/writing entire files. For large files, use `os.Open` + `bufio.Scanner` or `io.Copy` to avoid loading everything into memory.

Lesson 41: log/slog — Structured Logging (Go 1.21+)

log/slog is Go's standard structured logging package, introduced in Go 1.21. It replaces both the old log package and third-party libraries like logrus for most use cases. It provides leveled, key-value structured logging with pluggable handlers.

41.1 — C# ILogger vs Go slog

C# / .NET

```
// C# – ILogger (Microsoft.Extensions)
logger.LogInformation(
    "User {UserId} logged in from {IP}",
    userId, ipAddress);

logger.LogError(ex,
    "Failed to process order {OrderId}",
    orderId);

// Structured via message template
// Output depends on provider (Console,
// Serilog, NLog)
```

Go

```
// Go – log/slog
slog.Info("user logged in",
    "userId", userId,
    "ip", ipAddress)

slog.Error("failed to process order",
    "orderId", orderId,
    "error", err)

// Structured via key-value pairs
// Output depends on handler
// (TextHandler, JSONHandler)
```

41.2 — Configuring slog

```
import "log/slog"

// Development: human-readable text
textHandler := slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelDebug,
})

// Production: machine-parseable JSON
jsonHandler := slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level:      slog.LevelInfo,
    AddSource: true, // include file:line
})

// Set as default logger
logger := slog.New(jsonHandler)
slog.SetDefault(logger)

// Output: {"time":"...", "level":"INFO", "source":{"file":"main.go", "line":42},
//          "msg":"server started", "addr":":8080"}
```

41.3 — Logger with Context (Request-Scoped)

```
// Add persistent attributes to a logger
requestLogger := slog.Default().With(
    "requestID", requestID,
    "userID", userID,
```



```
)

// All subsequent logs include requestID + userID
requestLogger.Info("processing order", "orderId", 42)
// {"level":"INFO","msg":"processing order",
//  "requestID":"abc","userID":"u-123","orderId":42}

// Group related attributes
slog.Info("request completed",
    slog.Group("http",
        "method", r.Method,
        "path", r.URL.Path,
        "status", statusCode,
    ),
)
```

41.4 — Redacting Sensitive Data

```
// Implement slog.LogValuer to control how types are logged
type Token string

func (Token) LogValue() slog.Value {
    return slog.StringValue("REDACTED")
}

// Now logging a Token always shows "REDACTED"
slog.Info("auth", "token", Token("secret-abc-123"))
// {"level":"INFO","msg":"auth","token":"REDACTED"}
```

TIP: slog replaces logrus and zerolog for most applications. Use TextHandler in development, JSONHandler in production. For high-throughput services where every nanosecond matters, you can wire slog to a zap or zerolog backend via the Handler interface.

Lesson 42: time — Durations, Tickers, Timers & Formatting

Go's time package handles durations, timestamps, timers, and formatting. The formatting system is unique: instead of strftime patterns, Go uses a reference time (Mon Jan 2 15:04:05 MST 2006) as the template.

42.1 — C# DateTime/TimeSpan vs Go time

C# / .NET

```
// C# - DateTime & TimeSpan
var now = DateTime.UtcNow;
var dur = TimeSpan.FromSeconds(30);
var later = now.Add(dur);

var formatted = now.ToString(
    "yyyy-MM-dd HH:mm:ss");

Thread.Sleep(TimeSpan
    .FromMilliseconds(100));
```

Go

```
// Go - time.Time & time.Duration
now := time.Now()
dur := 30 * time.Second
later := now.Add(dur)

formatted := now.Format(
    "2006-01-02 15:04:05")

time.Sleep(100 * time.Millisecond)
```

42.2 — The Magic Reference Time

```
// Go's reference time: Mon Jan 2 15:04:05 MST 2006
// Each component is a specific number:
// Month: 01 (January)   Day: 02
// Hour: 15 (3 PM)      Minute: 04      Second: 05
// Year: 2006           Zone: MST

now := time.Now()
now.Format("2006-01-02")           // "2026-02-15"
now.Format("02/01/2006")          // "15/02/2026"
now.Format(time.RFC3339)          // "2026-02-15T10:30:00Z"
now.Format("3:04 PM")             // "10:30 AM"

// Parse
t, err := time.Parse("2006-01-02", "2026-02-15")
```

42.3 — Tickers and Timers

```
// Timer: fires once after duration
timer := time.NewTimer(5 * time.Second)
<-timer.C // blocks until timer fires

// Ticker: fires repeatedly at interval
ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop()

for {
```

```

    select {
    case t := <-ticker.C:
        fmt.Println("tick at", t)
    case <-ctx.Done():
        return
    }
}

// time.After for quick timeouts in select
select {
case result := <-resultCh:
    process(result)
case <-time.After(10 * time.Second):
    return errors.New("operation timed out")
}

```

42.4 — Phase 4 Summary: Standard Library Concept Map

C# / .NET Concept	Go Equivalent
ASP.NET Minimal API / Controllers	net/http + ServeMux (Go 1.22+)
System.Text.Json	encoding/json (+ v2 experimental)
[JsonPropertyName], [JsonIgnore]	Struct tags: `json:"name"`, `json:"-`"
Stream (abstract class)	io.Reader / io.Writer (interfaces)
xUnit / NUnit / MSTest	testing package (built-in)
[Theory] / [InlineData]	Table-driven tests
BenchmarkDotNet	testing.B benchmarks (built-in)
ADO.NET / EF Core	database/sql + driver
DbConnection pool	sql.DB (pool by default)
Razor Pages	html/template
System.IO.File	os.ReadFile / os.WriteFile
StreamReader	bufio.Scanner / bufio.Reader
Path.Combine	filepath.Join
ILogger / Serilog / NLog	log/slog (Go 1.21+)
DateTime / TimeSpan	time.Time / time.Duration
CancellationTokenSource + Timer	time.After in select

TIP: Go's standard library is unusually complete. Unlike C#.NET where you typically need NuGet packages for logging, HTTP routing, and JSON options, Go ships all of these built-in. The motto is 'a little copying is better than a little dependency.'